

JavaScript algajale

11IT · Veebirakendused · JavaScripti moodul, 1. nädal · konspekt

Mis on JavaScript?

Kujuta ette maja. HTML on seinad ja toad: mis kus asub. CSS on värvid ja mööbel: kuidas see välja näeb. **JavaScript on elekter**: tuli süttib, kui vajutad lülitit, ja uksekell heliseb, kui keegi nuppu vajutab. Ilma JavaScriptita saab lehte ainult vaadata, JavaScriptiga saab sellega midagi teha.

JavaScript on ainus programmeerimiskeel, mida brauser ise oskab käivitada. Seepärast on see kõige rohkem kasutatud keel veebis ja iga veebiarendaja puutub sellega kokku.

Kus JavaScripti proovida

Kõige kiirem koht on **konsool**. Ava ükskõik milline veebileht ja tee nii:

- vajuta F12 (Macis Cmd+Option+I), avanevad arendaja tööriistad;
- vali ülevalt vahekaart **Console**;
- kirjuta `2 + 2` ja vajuta Enter.

Konsool vastab kohe: 4. Proovi veel `"Tere, " + "maailm"`. Kõik selle konspekti lühikesed näited võid otse konsooli kirjutada ja vaadata, mis juhtub. Päris programmis kirjutatakse konsooli käsuga `console.log()`, näiteks `console.log("Tere!")`.

Muutujad: sildiga karbid

Muutuja on nagu karp, millele on kleebitud silt. Karpi saab panna väärtuse ja hiljem sildi järgi uuesti üles leida. JavaScriptis tehakse karp sõnaga **let** või **const**:

```
const nimi = "Mari"; // const: see väärtus enam ei muutu
let punktid = 10;     // let: see väärtus võib muutuda
punktid = punktid + 5; // nüüd on punktid 15
console.log(nimi, punktid);
```

Rusikareegel: kirjuta alati const. Kui hiljem selgub, et väärtust on vaja muuta, vaheta see let-iks. Rea lõpus olev semikoolon ; näitab, kus lause lõpeb. Kaks kaldkriipsu // alustavad kommentaari: see on märkus inimesele ja arvuti jätab selle vahele.

Väärtustel on tüübid. Esialgu piisab kolmest:

Tüüp	Mis see on	Näide
number	arv	17, 3.5, -2
string	tekst, alati jutumärkides	"Mari", "Tere!"
boolean	tõeväärtus: jah või ei	true, false

Teksti ja muutujaid saab mugavalt kokku panna tagurpidi ülakomade abil. `${ }` sisse kirjutatud muutuja asendatakse tema väärtusega:

```
const nimi = "Mari";
console.log(`Tere, ${nimi}!`); // Tere, Mari!
```

Tagurpidi ülakoma ` asub eri klaviatuuridel eri kohas. Kui märk kohe ei ilmu, vajuta pärast seda tühikut. Kui ei leia seda üldse, kopeeri see mõnest näitest.

Lõks: proovi konsoolis "5" + 3. Vastus ei ole 8, vaid "53". Jutumärkides 5 on tekst, ilma jutumärkideta arv, ja kui üks pool on tekst, paneb JavaScript tekstid lihtsalt kokku. Hoiatust ta sulle ei anna.

Tingimused: kui ..., siis ...

Programm peab tihti otsustama. Kui inimene on vähemalt 18, on ta täisealine, muidu mitte. Täpselt nii see ka kirja pannakse:

```
const vanus = 16;

if (vanus >= 18) {
  console.log("Täisealine");
} else {
  console.log("Alaealine");
}
```

Tingimus käib sulgudesse () ja see, mida teha, looksulgudesse { }. Looksulud on nagu karbi kaas: kõik, mis nende vahel on, kuulub kokku. Võrdlemiseks on need märgid:

Märk	Tähendus	Näide
===	on võrdne	vanus === 18
!==	ei ole võrdne	nimi !== "Mari"
> ja <	suurem kui ja väiksem kui	punktid > 40
>= ja <=	suurem või võrdne, väiksem või võrdne	vanus >= 18

Miks kolm võrdusmärki? Üks võrdusmärk = paneb karpi väärtuse. Kaks võrdusmärki == võrdleb, aga teeb seda lohakalt: tema arvates on tekst "5" ja arv 5 samad. Kolm võrdusmärki === võrdleb täpselt. Kasuta alati kolme.

Kui võimalusi on rohkem kui kaks, tuleb vahele **else if**. Arvuti kontrollib tingimusi ülevalt alla ja teeb esimese, mis sobib. Ülejäänud jäävad vahele:

```
const punktid = 82;

if (punktid >= 90) {
  console.log("Suurepärase!");
} else if (punktid >= 50) {
  console.log("Hea töö!"); // see käivitub
} else {
  console.log("Harjuta veel");
}
```

Funktsioonid: korduvkasutatavad retseptid

Funktsioon on nagu retsept. Kirjutad selle üks kord valmis, annad nime ja saad seda siis nii mitu korda kasutada, kui vaja. Retsepti koostisosad on **parameetrid** ja see, mis retseptist välja tuleb, antakse tagasi sõnaga **return**:

```
function tervita(nimi) {           // nimi on parameeter
  return `Tere, ${nimi}!`;        // tagastame valmis teksti
}

console.log(tervita("Mari"));     // Tere, Mari!
console.log(tervita("Jaan"));     // Tere, Jaan!
```

Varsti näed ka lühemat kirjaviisi, **noolfunktsiooni**. See teeb täpselt sama, lihtsalt vähema tähemärgiga: `const tervita = (nimi) => `Tere, ${nimi}!`;` Reactis kasutatakse seda väga palju.

Tsükel: tee sama asja mitu korda

Kui on vaja midagi korrata, näiteks kirjutada arvud 1 kuni 5, ei kirjuta keegi viit `console.log`-i. Selleks on tsükel:

```
for (let i = 1; i <= 5; i++) {
  console.log(i); // 1, 2, 3, 4, 5
}
```

Sulgudes on kolm osa, mis on eraldatud semikoolonitega:

- `let i = 1`: kust alustada. Loendur `i` algab ühest.
- `i <= 5`: kui kaua jätkata. Seni, kuni `i` on 5 või vähem.
- `i++`: mis teha pärast iga ringi. Suurenda `i`-d ühe võrra. `i++` on lühem kirjaviis sellest, mis on `i = i + 1`.

Tähte `i` kasutatakse loenduri nimena harjumusest, sõna „indeks“ järgi. Võid panna ka muu nime.

Massiivid: nimekirjad

Kui on vaja hoida mitut väärtust korraga, näiteks ostunimekirja, kasutatakse massiivi. See kirjutatakse nurksulgudesse:

```
const poed = ["leib", "piim", "juust"];

console.log(poed[0]); // leib, lugemist alustatakse nullist!
console.log(poed.length); // 3, nii palju asju on nimekirjas
poed.push("munad"); // lisab nimekirja lõppu

for (const asi of poed) { // käib nimekirja läbi ükshaaval
  console.log(asi);
}
```

Pane tähele, et esimene element on `poed[0]`, mitte `poed[1]`. Nii on see peaaegu kõigis programmeerimiskeeltes ja alguses ajab see kõiki segadusse.

Objektid: ühe asja kirjeldus

Objekt kirjeldab ühte asja mitme omadusega. Näiteks õpilasel on nimi ja klass. Objekt kirjutatakse looksulgudesse ja omadust loetakse punkti abil:

```
const opilane = { nimi: "Mari", klass: 11 };

console.log(opilane.nimi); // Mari
console.log(opilane.klass); // 11
```

Päris veebilehtedel on andmed peaaegu alati objektide nimekirj: õpilased, tooted, postitused. Järgmisel nädalal näed, et just sellisel kujul tulevad andmed ka serverist.

Kui oled Pythonit näinud

Kui sa pole Pythonit kirjutanud, võid selle osa vahele jätta. Kui oled, siis mõtted on samad, erineb peamiselt kirjaviis:

	Python	JavaScript
Muutuja	<code>x = 5</code>	<code>let x = 5;</code>
Väljund	<code>print("Tere")</code>	<code>console.log("Tere");</code>
Tingimus	<code>if x > 3:</code>	<code>if (x > 3) { ... }</code>
Funktsioon	<code>def liida(a, b):</code>	<code>function liida(a, b) { ... }</code>
List	<code>arvud.append(4)</code>	<code>arvud.push(4);</code>

JavaScript veebilehel

Päris lehel kirjutatakse JavaScript eraldi faili, näiteks script.js, ja see ühendatakse HTML-iga body lõpus:

```
<script src="script.js"></script>
</body>
```

Miks just lõpus? Brauser loeb HTML-i ülevalt alla. Kui skript oleks üleval, üritaks see elemente otsida enne, kui need lehel olemas on, ja ei leiaks midagi.

DOM: leht kui puu

Kui brauser HTML-i loeb, ehitab ta sellest puu. Seda puud nimetatakse **DOM**-iks. Ava arendaja tööriistades vahekaart **Elements** ja näed seda puud: iga element on oksake, mille sees võivad olla järgmised. JavaScript saab seda puud muuta ja muudatus ilmub kohe ekraanile.

Elemente otsitakse täpselt samade selektoritega nagu CSS-is: #id, .klass ja tagi nimi. See, mida CSS-i tunnis õppisid, tuleb siin kohe kasuks.

Mida teha	Kood
Leia üks element	<code>document.querySelector("#pealkiri")</code>
Leia kõik sobivad elemendid	<code>document.querySelectorAll(".tehtud")</code>
Muuda teksti	<code>pealkiri.textContent = "Tere!";</code>

Mida teha	Kood
Loe, mis on sisendväljas	<code>sisend.value</code>
Lisa või eemalda klass	<code>rida.classList.toggle("tehtud");</code>
Loo uus element	<code>const rida = document.createElement("li");</code>
Pane see lehele	<code>loend.append(rida);</code>
Reageeri klõpsule	<code>nupp.addEventListener("click", () => { ... });</code>

Sündmused: kui kasutaja midagi teeb

Sündmus on see, kui kasutaja midagi teeb: klõpsab, kirjutab või saadab vormi. `addEventListener` ütleb brauserile: „kui see juhtub, tee seda“. Siin on nupp, mis loeb oma klõpse:

```
const nupp = document.querySelector("button"); // leia nupp
let klikke = 0;                               // loendur algab nullist

nupp.addEventListener("click", () => {        // iga klõpsu peale:
  klikke = klikke + 1;                       // suurenda loendurit
  nupp.textContent = klikke;                 // ja näita seda nupul
});
```

Vormi puhul kasuta sündmust "submit" ja kirjuta esimeseks reaks `event.preventDefault()`. Muidu laadib brauser pärast vormi saatmist lehe uuesti ja kõik, mida su kood just tegi, kaob.

Vaata, ära arva. Kui midagi ei tööta, pane kahtlasesse kohta `console.log` ja vaata konsoolist, mis väärtus seal tegelikult on. See on kiirem kui koodi kümme korda üle lugeda.

Näide algusest lõpuni

Paneme kõik kokku. See väike leht küsib nime ja sünniaastat ning ütleb nupuvajutusel, kui vanaks inimene sel aastal saab. Loe kommentaarid läbi, iga rea juures on kirjas, mida see teeb.

```
<input id="nimi" placeholder="Nimi">
<input id="aasta" type="number" placeholder="Sünniaasta">
<button id="arvuta">Arvuta</button>
<p id="vastus"></p>

<script>
  // leiame lehelt vajalikud elemendid
  const nimi = document.querySelector("#nimi");
  const aasta = document.querySelector("#aasta");
  const vastus = document.querySelector("#vastus");

  // kui nuppu vajutatakse, arvutame vanuse
  document.querySelector("#arvuta").addEventListener("click", () => {
    const praegu = new Date().getFullYear(); // tänane aasta
    const vanus = praegu - Number(aasta.value); // tekst arvuks
    vastus.textContent =
      `${nimi.value} saab sel aastal ${vanus}-aastaseks.`;
  });
</script>
```

Miks `Number(aasta.value)`? Sisendvälja väärtus on alati tekst, isegi kui väli on `type="number"`. `Number()` teeb sellest arvu. Mäletad lõksu? `"17" + 1` annab `"171"`.

Kui midagi läheb valesti

Vead kuuluvad programmeerimise juurde, ka kogenud arendajatel. Need on konsoolis punasega ja siin on kõige sagedasemad:

Veateade	Mida see tähendab	Mida kontrollida
Cannot read properties of null	Elementi, mida otsisid, ei leitud.	Kas selektor on õige (id ees #)? Kas skript on body lõpus?
x is not defined	Sellise nimega muutujat või funktsiooni pole.	Trükiviga? Suur ja väike täht on erinevad.
Unexpected token	Kirjaviga, brauser ei saa lausest aru.	Puuduv sulg, looksulg või jutumärk. Vaata reanumbrit.
x is not a function	Kutsud välja midagi, mis pole funktsioon.	Kas nimi on täpselt õige? <code>addEventListener</code> ei ole sama mis <code>addListener</code> .

Veateate lõpus on faili nimi ja reanumber, näiteks `script.js:14`. Klõpsa sellel ja brauser näitab täpselt, kus viga on.

Lisaks, kui jõuad: map ja filter

Nimekirjaga saab teha ka nii, et iga elemendiga juhtub midagi (map) või jäetakse alles ainult sobivad (filter). Reactis on need väga levinud:

```
const arvud = [3, 12, 7, 20];  
const topelt = arvud.map((x) => x * 2);           // [6, 24, 14, 40]  
const suured = arvud.filter((x) => x > 10);      // [12, 20]
```

Sõnastik

Mõiste	Lihtsalt öeldes
muutuja	sildiga karp, kuhu saab väärtuse panna
funktsioon	nimega retsept, mida saab ikka ja jälle kasutada
tsükkel	kood, mis kordub mitu korda järjest
massiiv	nimekiri väärtustest, mis on järjekorras
objekt	ühe asja kirjeldus: nimi ja väärtus paarikaupa
konsool	brauseri koht, kus saab koodi proovida ja vigu näha
DOM	leht sellisena, nagu JavaScript seda näeb: elementide puu
sündmus	midagi, mida kasutaja teeb: klõps, sisestus, vormi saatmine

Kuhu see kõik viib?

Päris projektides ei kirjutata tänapäeval peaaegu kunagi tervet lehte puhta JavaScripti või TypeScriptiga. Kasutatakse raamistikke, näiteks Reacti. Aga raamistik on ehitatud täpselt nende asjade peale, mida sa praegu õpid: muutujad, funktsioonid, nimekirjad, DOM ja sündmused. Järgmisel nädalal näed oma tunnitöö peal ise, miks raamistikku üldse vaja on.

Järgmisel nädalal tuleb ka **TypeScript**: JavaScript, mis leiab paljud vead, näiteks sellesama "5" + 3 lõksu, üles juba enne, kui programm käima läheb.

Kontrolli ennast

- Mis vahe on let-il ja const-il?
- Mis on massiivi esimese elemendi number: 0 või 1?
- Mida teeb i++ ja mitu korda kordub tsükkel for (let i = 1; i <= 5; i++)?
- Miks kasutatakse võrdlemiseks kolme võrdusmärgi?
- Kirjuta funktsioon, mis tagastab arvu kahekordse.
- Miks paneme script-taggi body lõppu?
- Mida teeb event.preventDefault() vormi puhul?
- Mis tuleb välja, kui liita "5" + 3, ja miks?

Kõik vastused on selles konspektis. Kui mõni jääb segaseks, küsi järgmise tunni alguses.